

Resampling Techniques

Problem 16: Barnsley Fern and Markov Chains

Goal:

- Simulate a stochastic dynamical system using affine transformations
- Understand its interpretation as a **Markov chain**

Setup:

- Initial state:

$$\mathbf{x}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

- Update rule:

$$\mathbf{x}_{n+1} = A_i \mathbf{x}_n + \mathbf{b}_i$$

- At each step, choose $i \in \{1, 2, 3, 4\}$ randomly

Transformations

$$A_1 = \begin{bmatrix} 0 & 0 \\ 0 & 0.16 \end{bmatrix}, \quad \mathbf{b}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$A_2 = \begin{bmatrix} 0.85 & 0.04 \\ -0.04 & 0.85 \end{bmatrix}, \quad \mathbf{b}_2 = \begin{bmatrix} 0 \\ 1.6 \end{bmatrix}$$

$$A_3 = \begin{bmatrix} 0.20 & -0.26 \\ 0.23 & 0.22 \end{bmatrix}, \quad \mathbf{b}_3 = \begin{bmatrix} 0 \\ 1.6 \end{bmatrix}$$

$$A_4 = \begin{bmatrix} -0.15 & 0.28 \\ 0.26 & 0.24 \end{bmatrix}, \quad \mathbf{b}_4 = \begin{bmatrix} 0 \\ 0.44 \end{bmatrix}$$

Selection Probabilities

Choose transformation index i with:

- $P(i = 1) = 0.01$
- $P(i = 2) = 0.85$
- $P(i = 3) = 0.07$
- $P(i = 4) = 0.07$

Iteration

Repeat the update:

$$\mathbf{x}_{n+1} = A_i \mathbf{x}_n + \mathbf{b}_i$$

for $N \approx 30,000$ steps.

Task 16.1: Simulation

- 1 Generate the sequence:

$$\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_N$$

- 2 Plot the points (x_n, y_n)
- 3 Use at least $N = 30,000$

Expected: A fern-like structure emerges.

Task 16.2: Markov Property

Question

Why is this process a **Markov chain**?

- What is the state of the system?
- How is $\mathbf{x}_{n+1}$ determined?
- Does the update depend on $\mathbf{x}_{n-1}, \mathbf{x}_{n-2}, \dots$?

Task 16.3: Experiments

Modify the system and observe the effect:

- 1 Change the probabilities $P(i)$ (e.g., make them uniform)
- 2 Modify one of the matrices A_i
- 3 Change one of the vectors $\mathbf{b}_i$

Discussion

- How does the shape change?

Problem 17: Rejection Sampling

Goal:

- Sample from a Laplace distribution using rejection sampling
- Use a Normal distribution as the proposal

Target:

$$\pi(x) \propto f(x) = e^{-|x|}$$

- (Laplace distribution centered at 0)

Setup

Choose proposal:

$$g(x) = \mathcal{N}(0, \sigma^2)$$

- Easy to sample from
- Same support as target ($\mathbb{R}$)

Requirement

Find M such that:

$$f(x) \leq Mg(x), \quad \forall x$$

Task 17.1: Finding M

- ① Consider the ratio:

$$\frac{f(x)}{g(x)} = \frac{e^{-|x|}}{\frac{1}{\sqrt{2\pi\sigma^2}} e^{-x^2/(2\sigma^2)}}$$

- ② Simplify the expression
- ③ Find:

$$M = \sup_x \frac{f(x)}{g(x)}$$

Hint

Consider separately $x \geq 0$ and $x < 0$

Task 17.2: Rejection Sampling

Implement:

- 1 Sample $X \sim \mathcal{N}(0, \sigma^2)$
- 2 Sample $U \sim \text{Uniform}(0, 1)$
- 3 Accept if:

$$U \leq \frac{f(X)}{Mg(X)}$$

Output

Generate at least 10,000 accepted samples

Task 17.3: Verification

- 1 Plot histogram of samples
- 2 Overlay Laplace density:

$$\frac{1}{2}e^{-|x|}$$

Question

Does the histogram match the target distribution?

Task 17.4: Efficiency

- 1 Compute acceptance rate:

$$\frac{\text{accepted samples}}{\text{total trials}}$$

- 2 Compare with theoretical:

$$\frac{1}{M}$$

Task 17.5: Experiments

Modify the proposal variance σ^2 :

- Try different values of σ^2

Questions

- How does M change?
- How does acceptance rate change?
- Which σ^2 gives the best performance?

Task 17.6: Curse of Dimensionality

Extend the setup to higher dimensions.

- Target:

$$f(\mathbf{x}) = e^{-\|\mathbf{x}\|_1}$$

(Multivariate Laplace)

- Proposal:

$$g(\mathbf{x}) = \mathcal{N}(\mathbf{0}, \sigma^2 I_d)$$

Task

Compute:

$$M_d = \sup_{\mathbf{x}} \frac{f(\mathbf{x})}{g(\mathbf{x})}$$

for:

- $d = 1$
- $d = 2$
- $d = 3$

Task 17.6: Analysis

- Write the ratio:

$$\frac{f(\mathbf{x})}{g(\mathbf{x})} = \frac{e^{-\|\mathbf{x}\|_1}}{(2\pi\sigma^2)^{-d/2} e^{-\|\mathbf{x}\|_2^2/(2\sigma^2)}}$$

- Identify where the maximum occurs

Hint

The maximum occurs along coordinate axes (one component nonzero)

Task 17.6: Discussion

Questions

- How does M_d grow with dimension d ?
- What happens to the acceptance probability:

$$\frac{1}{M_d}?$$

Conclusion

Rejection sampling becomes inefficient in higher dimensions